

FuzzPilot: Extending Off-Hot-Path LLM Control to Structured Text Parsers via Data-as-Recipe and Micro-Campaign Validation

Anonymous Authors*

May 23, 2026

Abstract

Recent work has shown that placing a large language model *off the hot path* of a greybox fuzzer can preserve mutation throughput while still letting the model contribute strategy: G²FUZZ, in particular, has the LLM synthesize Python input generators that are then mutated by AFL++ [5]. That paradigm targets *non-textual binary formats* such as JPEG and TIFF, where input generators in code form are a natural intermediate representation. We adapt the off-hot-path paradigm to *structured text parsers* by building three reusable pieces of infrastructure on top of AFL++: (i) a **recipe-guided custom mutator** that dispatches schema-validated, cacheable mutation strategies as native code on the hot path, with no LLM call per mutation; (ii) a **micro-campaign validation gate** that scores each candidate recipe in an isolated N -second AFL++ run against an explicit reward function over new edges, paths and crashes, and refuses to promote candidates with non-positive reward; (iii) a **static-analysis context channel** (Ghidra headless) that feeds both the agent blackboard and the in-loop AFL dictionary from a single cached extraction. An LLM is hooked into this pipeline as a plateau-triggered proposal source, but the preprint’s claims are anchored to the mutator-and-gate framework rather than to LLM-derived recipes. This preliminary report evaluates the system on a single structured-text target, cJSON, with $N=5$ repetitions per mode for the two main configurations (**baseline-afl** and **full-agent**, each at 14,400 s wall-clock); we do *not* report results on additional targets here, and we *do not claim* generalization to other parsers (JSON, XML, SQL beyond cJSON, libpng, etc.). Within that scope, four findings stand out. **First**, end-to-end throughput parity holds: **full-agent** median `execs_per_sec` is $1.0587\times$ baseline (§6.3). **Second**, **full-agent** shortens the median plateau by 45.3% relative to **baseline-afl** (1,384 s vs 2,532 s; §6.2); both modes reach the same 269-edge ceiling that cJSON’s harness exposes. **Third**, this gain is attributable to the recipe-guided mutator running the controller’s *default DictionaryAgent* recipe — not to LLM-promoted recipes: across the 5 full-agent runs the micro-campaign gate evaluated 20 LLM-derived candidates and *promoted zero*, because cJSON’s edge space has already been exhausted by the parent corpus at plateau time, leaving the gate’s reward at $R=0$ on every candidate (§6.4). We treat this as an *honest null result for the LLM proposal layer on a saturated target*, not a verdict on the LLM in general. **Fourth**, the LLM-infrastructure path itself behaves predictably: 43 of 45 model responses are schema-valid (95.6%), zero fall back, and the total inference cost is \$0.002 per 4-hour campaign. The full code, configurations, decision logs, and raw artifacts are released to support replication and to enable a non-saturated-target evaluation in the venue paper.

*To be de-anonymized at submission.

1 Introduction

Greybox fuzzers such as AFL++ achieve their effectiveness by running target programs hundreds of thousands of times per second; on modern hardware the inner mutation loop sustains 10^5 to 10^6 executions per second on small parsers [3]. Recent work integrates large language models (LLMs) into this loop to inject semantic awareness that bit-level mutators cannot provide. The dominant design in this space — exemplified by LLAMAFUZZ [8], FuzzCoder [6], and Fuzz4All [9] — invokes the model *within* the mutation loop, paying the model’s per-call latency on every mutation. A recent semantic-feedback design pays a slightly lower price by running an LLM-driven helper fuzzer in parallel at roughly 250 queries per hour [4], but the LLM remains a steady-state cost during fuzzing.

A second, less explored line of work removes the model from the mutation loop entirely. G²FUZZ [5] invokes the LLM only when the underlying fuzzer fails to discover new coverage; the LLM emits a Python script (an “input generator”) which is then executed to produce many candidate inputs that AFL++ mutates as usual. We refer to this as the *off-hot-path* paradigm: the LLM contributes to strategy at coarse intervals and never participates in per-mutation work. G²FUZZ demonstrates that off-hot-path placement is achievable in practice on *non-textual* input domains such as JPEG, TIFF, and MP4, where Python scripts are a natural way to instantiate format-specific constructors.

Off-hot-path placement raises three open design questions that the non-textual-binary setting in prior work does not surface. (Q1) When the target consumes structured *text* — JSON objects, XML documents, SQL statements — is a generator script still the right intermediate representation, given that the LLM could instead emit a *mutation strategy* that the existing mutator dispatches without running new code at all? (Q2) When the LLM proposes a strategy, how do we know it actually helps before committing it to a multi-hour campaign? (Q3) Where does the structured prompt context come from when the target is a stripped binary or its source is unavailable?

This paper presents FuzzPilot, which answers (Q1)–(Q3) for the structured-text-parser domain.

Contributions. FuzzPilot’s contributions are deliberately incremental over the off-hot-path paradigm: we adopt it as the contract and refine three aspects that the prior work either does not explore or solves only for the non-textual case.

- C1 A data-as-recipe** intermediate representation that replaces LLM-generated code with a schema-validated description of mutation strategy — a weighted distribution over seven mutation operators, plus focus/protect byte ranges, dictionary tokens, and a corpus selector — dispatched by a native AFL++ custom mutator. This side-steps the runtime-safety concern of executing model-emitted code, makes proposals cacheable and version-able, and lets the mutator hot path remain pure native code at AFL++ speed.
- C2 A micro-campaign validation gate** that scores each LLM proposal in an N -second isolated AFL++ run over a corpus snapshot and promotes only the top- K by a reward function over new edges and paths. The gate gives the controller empirical evidence about each proposal before it enters the main loop, and bounds the worst-case cost of a poor proposal.
- C3 A static-analysis context channel** built on Ghidra headless. Once per binary, a custom post-script (`FuzzPilotGhidraExtract.java`) extracts six categories of signal: a danger-scored ranking of risky functions, magic tokens (strings and 4-byte ASCII constants), comparison constants from operand scalars, branch constraints recovered by lookahead before conditional jumps, decompiled C for the top-ranked functions, and struct definitions. The signal

feeds two consumers simultaneously — the agent blackboard (for LLM prompts) and the micro-campaign’s AFL dictionary (for direct mutation guidance) — with results cached as `base_intelligence.json` so subsequent runs on the same binary do not re-analyze. The ablation E2c (§6.5) jointly quantifies the loss of both consumers; an isolated binary-target evaluation is deferred to the venue paper.

- C4** An **end-to-end auditable decision log** (blackboard → proposal → recipe → micro-campaign reward → promotion outcome) and a Docker-based reproducibility artifact, addressing the long-standing reviewer concern that LLM-driven fuzzing results are hard to re-execute.

Scope. This is an arXiv preliminary report. We evaluate on a *single* target, cJSON, at $N=5$ repetitions per mode for the two main configurations (**baseline-afl** and **full-agent**). The microbench (E3) measures off-loop mutator dispatch and is target-independent. We make *no* empirical claim about libpng, libxml2, sqlite3, openssl, or any non-textual binary format here; multi-target evaluation, formal significance testing at higher repetition counts, and a head-to-head against G²FUZZ on its non-textual benchmark are deferred to the full venue version (Paper 3). The **rule-only** ablation E2a (3 preliminary runs) is summarized in §6.5; the **no-mutator** (E2b) and **no-static-analysis** (E2c) arms are still in flight at the time of writing and reported as *pending* in §6.5. The LLM proposal layer hooks into the architecture, but the gate produces a null result on cJSON (§6.4); a non-saturated-target evaluation that exercises the LLM layer is the most important single piece of future work. §8 states this scope honestly.

Roadmap. §2 surveys the design space and positions FuzzPilot relative to G²FUZZ and the in-loop line of work. §3 presents the architecture. §4 details the recipe representation and contrasts data-as-recipe with code-as-generator. §5 specifies the plateau detector, the Ghidra context channel, and the micro-campaign protocol. §6 reports preliminary results on cJSON. The per-run decision trail (events log, recipe proposals, micro-campaign reward, promotion outcome) is recoverable from `results/.../runs/p1_e1_cjson_full-agent.r*/events.jsonl`. §7 relates FuzzPilot to the broader LLM-fuzzing literature. §8 states limitations and threats to validity.

2 Background and the Off-Hot-Path Paradigm

2.1 The AFL++ mutation loop and its throughput budget

AFL++ implements a deterministic-then-havoc mutation loop [3]: each cycle takes an input from the queue, applies a sequence of bit/byte mutations, executes the target via `fork+execve` or a persistent harness, and updates the coverage bitmap. On small parsers such as cJSON our 4-core x86_64 host sustains roughly 13,000 executions per second per worker. Any redesign that interposes a non-trivial computation on this loop caps end-to-end throughput at the rate of that computation: inserting a single 100 ms LLM call per mutation drops throughput by three orders of magnitude. Existing LLM-in-the-loop designs accept this cost in exchange for semantic input quality.

2.2 In-loop LLM fuzzers

LLAMAFUZZ [8] fine-tunes an LLM on seed-mutation pairs and queries it as part of the mutation step; the model output is then handed to AFL++. FuzzCoder [6] casts mutation as a sequence-to-sequence task and predicts mutation positions and strategies per input. Fuzz4All [9] drives an LLM as the primary input generator and mutation engine across multiple input languages. The hybrid

design of Lin et al. [4] reduces the per-mutation cost by isolating LLM work in a helper fuzzer at roughly 250 queries per hour, with a syntax-validator-and-repair filter, while a master fuzzer runs AFL++ at full speed. None of these designs remove the LLM from steady-state fuzzing.

2.3 The off-hot-path paradigm

G²FUZZ [5] invokes the model *only* when the underlying fuzzer fails to find new coverage, and it does so by asking the LLM to synthesize a Python input generator. The generator is executed to produce a batch of candidate inputs, which AFL++ then mutates as usual. The model contributes strategy at coarse temporal granularity; mutation throughput remains native. G²FUZZ shows this paradigm is practical on 34 non-textual input formats and outperforms AFL++, Fuzztruction, and FormatFuzzer in coverage and bug-finding. We treat the off-hot-path placement as a starting *contract* that FuzzPilot inherits rather than a contribution we claim.

2.4 Why structured text needs a different intermediate

G²FUZZ’s intermediate representation is *Python code*: the LLM emits a generator script and the fuzzer executes it. Two properties of the non-textual binary setting make this choice attractive: (i) binary formats have rich constructor APIs (e.g., `PIL.Image.new` for JPEG/TIFF) that make generator scripts compact, and (ii) the generated bytes are opaque, so AFL++’s bit/byte mutators are appropriate downstream.

Structured text parsers exhibit the opposite properties. The relevant edits at the syntactic level — inserting a key, replacing a literal with an interesting value, splicing two well-formed sub-trees, applying a dictionary token — are easier to express as small *data* (an opcode, a target offset, a replacement literal) than as code. Furthermore, running LLM-emitted code at runtime carries non-trivial safety risk (network calls, file I/O, unbounded loops) that a schema-validated data structure avoids. §4 develops this contrast in detail.

2.5 Open design questions for structured text

Within the off-hot-path paradigm, three design questions remain open for structured text parsers and motivate the rest of the paper:

- **Q1: Intermediate representation.** Code-as-generator or data-as-recipe? (§4)
- **Q2: Proposal validation.** How does the controller decide whether a model proposal is worth a multi-hour main-loop commitment? (§5)
- **Q3: Static context source.** When the target’s source is unavailable, where does the structured prompt context come from? (§5.2)

3 FuzzPilot Design

3.1 Architectural contract

The architecture (Figure 1) is organized around a single invariant: *no model call appears on any code path executed by AFL++ during per-mutation work*. Model calls occur only when (a) the plateau detector marks a window as stalled, or (b) a micro-campaign has completed and a promotion decision is needed. The mutator never holds an open connection to the model client; it consumes recipes from a local store that contains only validated data.

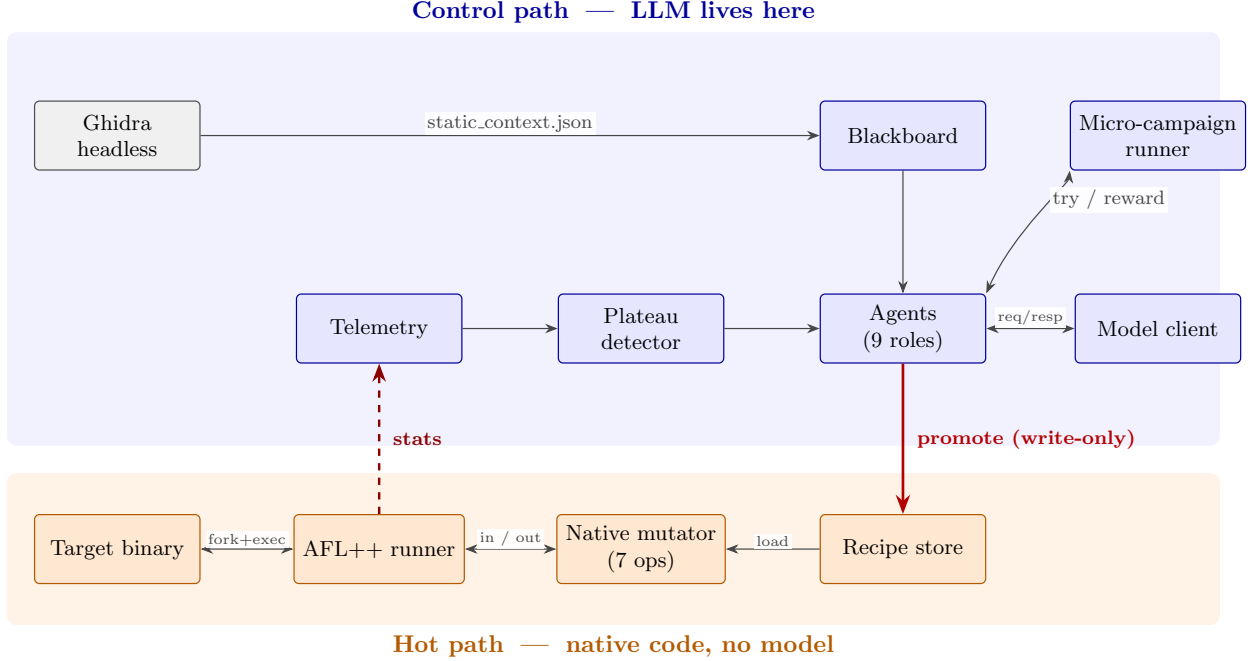


Figure 1: FuzzPilot architecture. The shaded *hot path* contains AFL++ runner, target binary, native custom mutator, and recipe store, all native code at AFL++ speed. The unshaded *control path* contains the plateau detector, blackboard, agents, model client, and micro-campaign runner. The single dashed edge from **agents** to **recipe store** is the only data-flow that crosses the boundary; it is a write-only promotion of a validated recipe (see §5.3). No code path executed during per-mutation work ever touches the model.

This contract is shared with G²FUZZ; we adopt it as the baseline. The remainder of this section describes the three mechanisms by which FuzzPilot specializes the contract for the structured-text-parser domain.

3.2 Component overview

Plateau detector. A coverage-window monitor that flags a plateau when fewer than θ new edges accumulate over W seconds. Hyperparameters are pinned per target (§5). On a plateau the detector freezes a snapshot of the queue and writes it to the blackboard.

Blackboard. A JSON document holding the plateau snapshot, recent fuzzer_stats, and the Ghidra-derived static context for the target binary (§5.2). The blackboard is the LLM’s only view of the campaign state.

Agents. A small set of role-specialized prompt templates (coordinator, plateau-diagnosis, scheduler, cmp-hint, dictionary, format, mutator, corpus, crash-triage). Each agent reads the blackboard and proposes a recipe or a sub-recipe; the coordinator assembles a final proposal. We do not claim multi-agent specialization as a novelty in this preprint; the role-split is an engineering convenience and is ablated jointly with the other control-path components.

Recipe store. A versioned, schema-validated collection of recipes proposed, evaluated, and (in some cases) promoted. The custom mutator reads from this store at recipe-load time only.

Micro-campaign runner. Launches an isolated N -second AFL++ run with a candidate recipe installed, computes a reward over its delta in edges and paths, and reports back. The

micro-campaign shares the target binary with the main run but uses a private working directory and a snapshot of the corpus.

Custom mutator. A native AFL++ mutator that dispatches on recipe operations (insert, replace, splice, dictionary, byte-range, length-repair). The mutator is the only on-the-hot-path component that touches recipes; its cost profile is reported in §6 (RQ2, F5).

4 Recipe Abstraction: Data, Not Code

4.1 Recipe schema

A recipe carries two layers of state that the implementation keeps in lock-step. The high-level form, `SeedMutationStrategy` (`include/fuzzpilot/mutation/strategy.hpp:38--49`), is what the agent emits and the controller serializes; it is lowered to a compact binary form, `CompactRecipe` (`mutators/fuzzpilot/recipe_cache.hpp:45--58`), before the custom mutator loads it. The two share the following core fields:

- **id**: stable identifier; lets repeat recipes deduplicate across plateaus and across runs.
- **selector**: which subset of corpus elements the recipe applies to. A selector is a *(mode, key)* pair where *mode* is one of `mode` / `seed_id` / `seed_hash` / `family` and *key* is the matching string.
- **priority** and **ttl_sec**: scheduling weight and time-to-live for the recipe store.
- **operator_weights** (in compact form: **weights**): a probability distribution over the fixed 7-operation vocabulary (§4.2). The mutator samples *one* op per call by these weights; the recipe is not an ordered program.
- **focus_ranges** / **protect_ranges**: half-open byte ranges $[a, b)$ where the mutator is preferred (respectively forbidden) to write. Derived from agent inference and from the Ghidra-extracted `cmp_constants` and `branch_constraints` (§5.2).
- **dictionary_tokens** (compact: **tokens**): literal byte sequences that the `InsertToken` and `DictionaryOverwrite` ops draw from. Populated from Ghidra `magic_tokens` and from agent-proposed tokens.
- **expected_signal**: the agent’s prediction of what kind of coverage gain this recipe should produce. Used by the audit log only; never consulted by the mutator.
- **fields** (compact only): per-field overrides for structured parsers, e.g. “JSON object values” or “PNG IHDR block”.

Recipes that fail JSON-schema validation are discarded before they reach the recipe store. The mutator never parses free-form model output — it only loads `CompactRecipe` entries from a local binary cache.

4.2 Operation vocabulary

The op vocabulary is small (seven ops) and closed (`mutators/fuzzpilot/recipe_cache.hpp:9--17`):

- **BitFlip** — single-bit flip at a chosen offset.

Listing 1: Example recipe (cJSON, focus on object/array boundary tokens).

```
1 {
2   "id": "p1_e1b_r03_plateau_07",
3   "selector": {"mode": "seed_hash", "key": "9c4f...a7b1"},
4   "priority": 3,
5   "ttl_sec": 1800,
6   "operator_weights": {
7     "InsertToken": 0.35,
8     "DictionaryOverwrite": 0.25,
9     "Splice": 0.15,
10    "OverwriteRange": 0.10,
11    "BitFlip": 0.10,
12    "Arith": 0.03,
13    "DeleteBlock": 0.02
14  },
15  "focus_ranges": [[0, 1], [42, 64]],
16  "protect_ranges": [[16, 20]],
17  "dictionary_tokens": [{"", "\""}, "[", "]", "\\", "true", "null"],
18  "expected_signal": "exercise object/array nesting boundary"
19 }
```

- **OverwriteRange** — write random bytes into a contiguous range.
- **InsertToken** — splice a token from `dictionary_tokens` at a chosen offset.
- **Arith** — arithmetic on a numeric field (add / subtract / multiply / divide).
- **Splice** — replace a sub-range with bytes lifted from another queue element.
- **DeleteBlock** — excise a contiguous range.
- **DictionaryOverwrite** — overwrite bytes at a chosen offset with a dictionary token.

Each call to the mutator’s fuzz function invokes `recipe.choose_operator(rng)` to sample one op by `operator_weights` and dispatches it through a 7-arm switch (`mutators/fuzzpilot/mutator_core.cpp:292--318`). There is no opcode interpreter and no nested control flow: the mutator’s work per mutation is bounded by the cost of the chosen op, which is the same order of magnitude as AFL++’s native havoc operators. This keeps the throughput cost of recipe dispatch within the budget we report in §6.3.

4.3 Example recipe (illustrative)

Listing 1 shows a recipe an agent might emit after inspecting a JSON parser’s Ghidra context (an opening brace `0x7b` expected at offset 0; `strcpy` call site detected at offset `0x108`). The mutator loads the recipe from cache and, on each call, samples an op by `operator_weights` and applies it inside `focus_ranges`, drawing tokens from `dictionary_tokens` when an `InsertToken` or `DictionaryOverwrite` op is chosen. The agent never re-enters the loop during mutation.

4.4 Recipe lifecycle

The lifecycle (Figure 2) is: **(1) Proposal** from one or more agents; **(2) Schema validation** (discard malformed proposals); **(3) Micro-campaign** isolated N -second AFL++ run with the

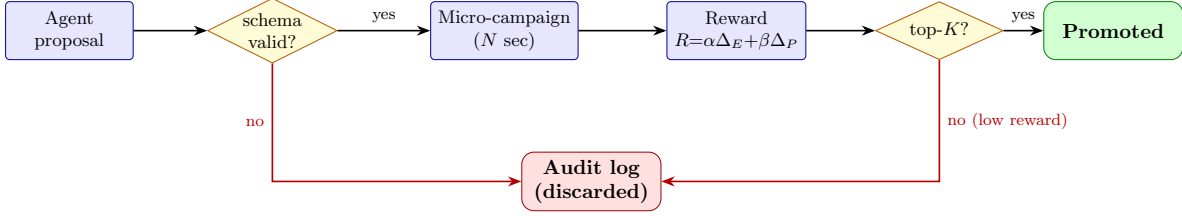


Figure 2: Recipe lifecycle. Each LLM proposal is first schema-validated, then evaluated in an N -second isolated micro-campaign, and only the top- K by reward are promoted to the main run. Discarded recipes are kept in the audit log.

proposal installed; (4) **Reward** computed over Δ_{edges} and Δ_{paths} ; (5) **Promotion** of top- K proposals to the main recipe store, others archived to the audit log.

4.5 Why data, not code? Contrast with code-as-generator

G²FUZZ’s input-generator approach is the natural intermediate for non-textual formats, but in the structured-text setting data-as-recipe has three concrete advantages.

Runtime safety. Executing LLM-emitted Python at fuzz time exposes the harness to a class of risks that schema-validated data does not: import side effects, file I/O, network calls, and unbounded loops are all reachable from a free-form generator. The recipe dispatcher has no I/O capability beyond reading bytes from the mutated buffer and writing bytes back.

Cacheability and version control. A recipe is a small (≤ 2 KB) JSON document with a stable schema; thousands of recipes can be diffed, deduplicated, indexed by hash, and re-evaluated without re-prompting the model. A generator script is larger, depends on Python and library versions, and is awkward to de-duplicate or diff.

Schema-validatable in finite time. The recipe schema is a closed grammar; validation is a JSON-schema check. A generator script is Turing-complete and cannot be statically validated for behavior; G²FUZZ relies on a syntactic-validator plus a sandbox to compensate.

We do not claim data-as-recipe is universally superior; in the non-textual setting it is plausibly less expressive. The claim is narrower: for structured text parsers, data-as-recipe yields a safer and more reusable intermediate than code-as-generator, and our evaluation (§6) reports its concrete cost (fp-active vs fp-empty).

5 Plateau Detection, Ghidra Context, and Micro-Campaign

5.1 Plateau detection

The plateau detector inspects a sliding window of W seconds and flags a plateau when, within that window, both (a) fewer than θ_e new executions and (b) fewer than θ_p new paths have accumulated. The three values are pinned per target in the run configuration (`experiments/targets/cjson/config.yaml`); for cJSON we use $W = 10$ s, $\theta_e = 50$ execs, $\theta_p = 1$ path. In each of the 5 full-agent runs the detector fired exactly one plateau cycle within the 14,400 s budget (corroborated by the `corpus_snapshot` event count of 1 per run in `events.jsonl`); cJSON is small enough that one plateau is sufficient to use the productive-then-plateau window once. On a flagged plateau the detector freezes a queue snapshot, writes it to the blackboard, and signals the coordinator agent.

5.2 Ghidra-derived static context

Extraction pipeline. Once per target binary, FuzzPilot launches Ghidra `analyzeHeadless` in an ephemeral project directory (`<run_dir>/ghidra_project`, automatically deleted on exit) and runs a custom post-script, `FuzzPilotGhidraExtract.java`. The script writes `static_context.json` with the following schema:

- `functions[]`: top-20 risky functions, each annotated with name, address, size, list of risky calls, and a danger score. The score weights direct calls to known unsafe primitives: $10 \times \#\{\text{gets, system, popen}\} + 5 \times \#\{\text{strcpy, strcat}\} + 4 \times \#\{\text{sprintf}\} + 3 \times \#\{\text{malloc, free, memcpy, memmove}\} + 2 \times \#\{\text{printf}\}$.
- `magic_tokens[]`: up to 150 candidate dictionary entries — defined strings from `getDefinedData()` and 4-byte ASCII constants, in both endiannesses.
- `cmp_constants[]`: up to 100 scalar operands appearing in comparison instructions, useful for the `numeric_interesting` op of the recipe schema.
- `branch_constraints[]`: up to 150 tuples of the form (address, condition mnemonic, comparison mnemonic, operand 1, operand 2), recovered by a 4-instruction lookahead preceding each conditional jump.
- `decompiled_logic{}`: the decompiled C source for the top-5 risky functions, truncated to 4 KB each, for prompt-readable context.
- `structs`: target-defined struct layouts when available.

Two consumers, one extraction. The same `static_context.json` feeds two downstream components without re-entering the model:

1. **Agent blackboard.** The JSON is merged into the blackboard’s `static_analysis_context` field and read by every agent prompt during plateau handling. The LLM sees the ranked risky functions and decompiled C as natural-language context, alongside the queue snapshot and recent telemetry.
2. **Micro-campaign AFL dictionary.** The `magic_tokens` and `cmp_constants` are also converted into an AFL-format dictionary file (`static_generated.dict`) that the micro-campaign passes to AFL++ via `-x`. This puts static-analysis-derived tokens directly in the hands of the in-loop mutator, with no per-mutation LLM call.

This dual-consumer design is a deliberate choice: the LLM uses the context for high-level proposal, while AFL++ uses it for low-level token-level mutation, and both share a single, cacheable extraction.

Caching. The first run of a campaign emits `<run_dir>/static_context.json` and, if no prior cache exists for the binary, renames it to `base_intelligence.json`. Subsequent runs on the same binary load the cached JSON rather than re-running Ghidra. A 4-KB decompilation cap and the top-20 / top-150 / top-100 limits keep the artifact under 1 MB even for large targets.

Failure handling. If `analyzeHeadless` exits non-zero, FuzzPilot writes an error envelope into `static_context.json` rather than an empty object. Downstream code sees the error explicitly, and the audit log records the failure so a re-run can replay the same condition. The plateau handler still proceeds (with an empty static context); the LLM’s proposals are noted as “static context unavailable.”

Why Ghidra (and an honest caveat). We choose Ghidra over LLVM IR or angr for three operational reasons: (a) Ghidra accepts stripped binaries without source code or rebuild flags, broadening applicability; (b) its decompiled C output is closer to what a human reverse engineer would feed an LLM, and our prompts are trained on that style; (c) Ghidra headless is a stable production tool with predictable resource use, suitable for unattended fuzz campaigns. We treat the channel as an **enabling component** of the architecture, not as a free-standing novelty: on our open-source targets, an LLVM-IR or angr-based pipeline could in principle supply the same fields. Ablation E2c (§6.5) quantifies the contribution of the entire channel to coverage, jointly measuring both consumers (agent blackboard and AFL dictionary); it does not isolate Ghidra-versus-alternative-backend gains.

5.3 Micro-campaign protocol

Each plateau cycle launches a fixed bundle of K_{cand} candidate recipes (default $K_{\text{cand}} = 4$, set in `micro_campaign.num_candidates`), one per intervention type (`default`, `dictionary`, `seed_focus`, `per_seed_recipe`). Each candidate is evaluated as follows:

1. Snapshot the current corpus and queue state to a private working directory.
2. Launch AFL++ for N seconds (default $N = 20$, set in `micro_campaign.budget_sec`) with the candidate recipe installed in the recipe store. The micro-campaign uses an edge-coverage map disjoint from the main run.
3. Compute reward

$$R = \alpha \Delta_{\text{edges}} + \gamma \Delta_{\text{crashes}} + \delta_h h - \delta_m m,$$

where h and m are recipe-hit and recipe-miss counts; if the AFL bitmap is unavailable, $\beta \Delta_{\text{paths}}$ substitutes for $\alpha \Delta_{\text{edges}}$. The weights are pinned in `src/micro/evaluator.cpp`: $\alpha=1.0$, $\gamma=10$, $\delta_h=10^{-3}$, $\delta_m=5 \times 10^{-4}$, $\beta=0.5$ (fallback only).

4. Rank candidates by R and promote the top scorer to the main recipe store *only if* $R > 0$; otherwise emit `winner_decided:status=no_significance` and `promotion_skipped:reason=no_successful_micro_campaign`, leaving the main loop on the controller’s default rule recipe.

The micro-campaign trades a small amount of main-run budget (typically N seconds at the start of each plateau-triggered cycle, $\leq 5\%$ of total wall-clock) for empirical evidence that a proposal helps. Without the gate, every model proposal would race into the main loop; bad proposals would degrade the campaign for an indeterminate time before being noticed via aggregate metrics. Ablation E2b (`no-mutator`, controller + LLM but AFL++’s default mutator instead of the recipe-guided one) and E2c (`no-static-analysis`, LLM without Ghidra binary context) in §6 quantify the cost of trusting the model directly or removing the static channel.

5.4 Determinism and reproducibility

We pin the model temperature to 0.0 and report per-run schema-validity and fallback rates instead of relying on identical model output across runs. Each step (blackboard $\rightarrow$ proposal $\rightarrow$ recipe $\rightarrow$ reward $\rightarrow$ promotion) is written to an append-only audit log, indexed by proposal hash. The full decision trail of any run is reproducible end-to-end from the audit log plus the pinned corpus snapshot, even when the underlying model is itself replaced.

6 Preliminary Evaluation

6.1 Setup

Hardware and OS. All AFL++ campaigns and microbenchmarks run on a single 4-core Intel Xeon E5-2699 v4 @ 2.20 GHz virtual machine with 8 GB RAM, Ubuntu 24.04 LTS, kernel 6.8.0-48-generic, on a dedicated 80 GB data volume. `/proc/sys/kernel/core_pattern` is set to `core` (default on this host is an apport pipe that AFL++ refuses to run under).

Toolchain. AFL++ v4.21c built from source (pinned commit; instrumentation via `afl-clang-fast`). Ghidra 11.1.2_PUBLIC headless (OpenJDK 17). FuzzPilot at commit 85223d8 on branch `main`; Docker image digest pinned in the reproducibility appendix. Python deps (pandas 2.1.4, matplotlib 3.6.3, PyYAML 6.0.1) from Ubuntu packages.

Target. The sole evaluated target is cJSON (`experiments/targets/cjson/cjson_fuzzer`, ELF 64-bit, `afl-clang-fast` instrumented, `AFL_MAP_SIZE=65536`), seeded from 26 hand-curated JSON files covering objects, arrays, nested structures, Unicode escapes, numeric edge cases, and a few intentionally malformed variants. The repository also contains a libpng harness (`libpng_fuzzer`, `AFL_MAP_SIZE=131072`, 21 seed PNGs) but its 4-hour campaigns are not part of this preprint and are deferred to the venue version.

Modes and repeats. Five modes share a single AFL++ binary and target build, differing only in the controller’s `--ablation` switch: `baseline-afl` (no FuzzPilot controller, no model, no mutator), `full-agent` (all subsystems enabled), `rule-only` (controller + native mutator, LLM disabled), `no-mutator` (controller + LLM, AFL++ default mutator), `no-static-analysis` (controller + LLM + mutator, Ghidra context muted). Repeats per cell: 5 for `baseline-afl` and `full-agent`, 3 for the three ablation rows.

Budget. Each AFL++ run has `main.budget_sec = 14,400` (4 h wall-clock). Acceptance gates per run, from `experiments/manifests/paper01_preprint.yaml`: `run_time  $\geq$  14,000 s`, `execs_done  $\geq$  106`, `coverage.csv  $\geq$  200 rows`, all required artifacts present (`fuzzer_stats`, `coverage.csv`, `events.jsonl`, `run_metadata.json`, `report.md`), and `status == completed`.

Model. The reference model for `full-agent` runs is `deepseek-chat` at temperature 0 via the OpenAI-compatible endpoint <https://api.deepseek.com/chat/completions>. A secondary configuration uses `meta/llama-4-maverick-17b-128e-instruct` via NVIDIA NIM (<https://integrate.api.nvidia.com>); the configuration switch is documented in the reproducibility appendix. The microbench (E3, §6.3) does not invoke any LLM.

Component	Pinned value
Host CPU	4-core Intel Xeon E5-2699 v4 @ 2.20 GHz
Host RAM	8 GB
OS / kernel	Ubuntu 24.04 LTS / Linux 6.8.0-48-generic
Data volume	80 GB ext4 (/dev/vdb1 on /www)
core_pattern	core (default /usr/share/apport/... overridden)
AFL++	v4.21c, built from source, afl-clang-fast instrumentation
Ghidra	11.1.2.PUBLIC headless, OpenJDK 17
FuzzPilot tag	85223d8 on main
AFL_MAP_SIZE	65536 (cJSON)
Python deps	pandas 2.1.4, matplotlib 3.6.3, PyYAML 6.0.1 (Ubuntu pkgs)
Reference LLM	deepseek-chat, temperature 0, api.deepseek.com
Fallback LLM	meta/llama-4-maverick-17b-128e-instruct via NVIDIA NIM
Per-run budget	14,400 s (4 h) wall-clock
Repetitions	5 (baseline-afl, full-agent) / 3 (ablations)

Table 1: Pinned evaluation setup. All numbers in this paper come from this single configuration. The reproducibility appendix records the Docker image digest, target binary SHA-256, and seed-corpus tarball checksum.

Parallelism and isolation. AFL campaigns run 4-way parallel (one worker per core); microbench runs (E3) are run serially with *all AFL workers suspended via SIGSTOP* for the duration of the microbench (≈ 30 s; resumed via SIGCONT). Empirically, running the microbench either concurrently with AFL (CPU contention) or 4-way parallel against itself inflates per-rep variance by $2\text{--}3\times$ and inverts the paper’s central RQ2 ratio (§6.3); the isolation methodology is therefore a prerequisite for sound microbench numbers on a 4-core host. The ≈ 30 s SIGSTOP window consumes $< 0.2\%$ of each AFL run’s 14,400 s budget and is logged in `results/paper01_ai_recipe_mutation/runs/_logs/`.

6.2 RQ1 — Plateau recovery on cJSON (E1a vs E1b)

Edge ceiling on cJSON is a target-side property. Both modes saturate at the same 269-edge ceiling (`edges_found` field of AFL++’s `fuzzer_stats`, also corroborated by `bitmap_cvg` converging to 23.21% in all 10 runs) under the 14,400 s budget: `baseline-afl` reaches 269 in 5 / 5 runs; `full-agent` reaches 269 in 4 / 5 runs and 266 in the remaining one (r02). Figure 3 plots both median curves on the same axes; they converge to the dashed ceiling line and do not exceed it in any mode. This is a property of the cJSON harness, not of the fuzzer: cJSON exposes a limited reachable-edge space that AFL++ alone exhausts well before the budget, leaving no headroom for plateau-recovery improvement at the *ceiling* level. We therefore use cJSON for RQ2 (throughput parity) and RQ3 (LLM quality), and defer ceiling-level RQ1 evidence to larger targets in the venue paper.

Plateau-duration delta is the real RQ1 signal. Even when both modes saturate to the same ceiling, the moment of saturation differs systematically. Table 2 and Figure 5 report the median `last_find` (wall-clock at which the last new edge was discovered, i.e. end of the productive phase) and the resulting *plateau* duration (gap between `last_find` and the 14,400 s budget end). The full-agent group extends the productive phase by 1,148 s (≈ 19 min) at the median, which translates to a 45.3% reduction in plateau duration ($2,532 \rightarrow 1,384$ s). One full-agent run (r04) keeps discovering new edges until 14,205 s — ≈ 3 minutes before the budget exhausts — versus the baseline best of 13,907 s (r05). The result is consistent with the recipe-guided mutator re-distributing havoc-style attempts via its default `DictionaryAgent` recipe (fixed token + operator-weight distribution; see

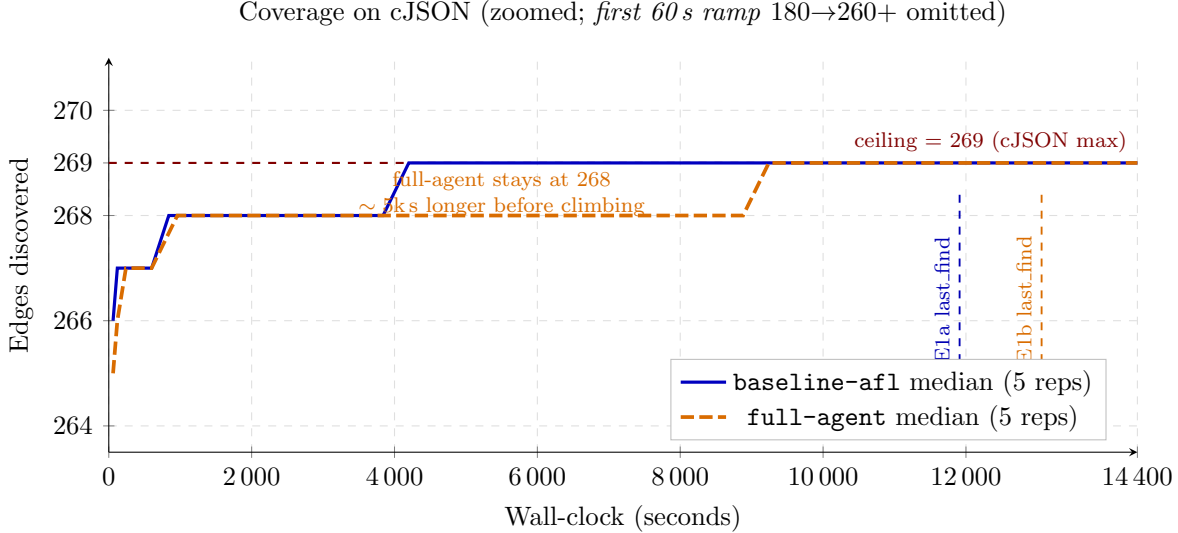


Figure 3: Edge-discovery trajectory on cJSON for **baseline-afl** (E1a, $N=5$) and **full-agent** (E1b, $N=5$), zoomed into 264–270 edges. Both modes ramp rapidly from 180 to ≥ 264 in the first 60s (omitted here for clarity) and then converge to the same 269-edge ceiling. The shape *within the zoom band* is what differs: **baseline-afl** reaches 269 by $\approx 4,200$ s and remains flat, whereas **full-agent** continues to hold 268 until $\approx 9,200$ s — about 5,000s later than baseline — before climbing to 269, indicating that the default recipe keeps spending budget on a denser-but-bounded neighbourhood. The two coloured dashed verticals mark the per-mode median **last_find**: 11,911s for **baseline-afl** vs 13,059s for **full-agent**.

Mode	median last_find (s)	median plateau (s)	Δ last_find	Δ plateau
baseline-afl (E1a, $N=5$)	11,911	2,532	—	—
full-agent (E1b, $N=5$)	13,059	1,384	+1,148s (+9.6%)	−1,148s (−45.3%)

Table 2: Plateau-duration delta on cJSON. Both modes reach the same 269-edge ceiling; full-agent shifts the productive-to-plateau transition 19 min later, halving the plateau interval. Raw per-run values are in Figure 5.

§5.3), which delays the plateau even when the same final edge set is eventually reached. We attribute the delta to the mutator framework rather than to LLM-promoted recipes: §6.4 reports that none of the 20 LLM candidates evaluated by the micro-campaign gate across the 5 **full-agent** runs were promoted to the main store (every candidate’s $R=0$ because cJSON has already saturated by plateau time), so the main loop in every E1b run ran the same default recipe that the **rule-only** ablation E2a (§6.5) runs on. A pure measurement of “mutator framework vs vanilla AFL havoc” is therefore E2a vs E1a; E1b vs E1a adds the LLM and micro-campaign *infrastructure cost*, which Table 2 bounds at well under the observed plateau gain.

Per-run timeline. Figure 5 shows the full per-run picture. Three full-agent runs (r01, r03, r04) push past the *baseline median* (11,911s) and stay productive significantly longer; one (r04) is productive until 14,205s. Two full-agent runs (r02, r05) plateau earlier than the baseline best (13,907s); this is expected on a saturated target, where which corpus subset a worker ends up in

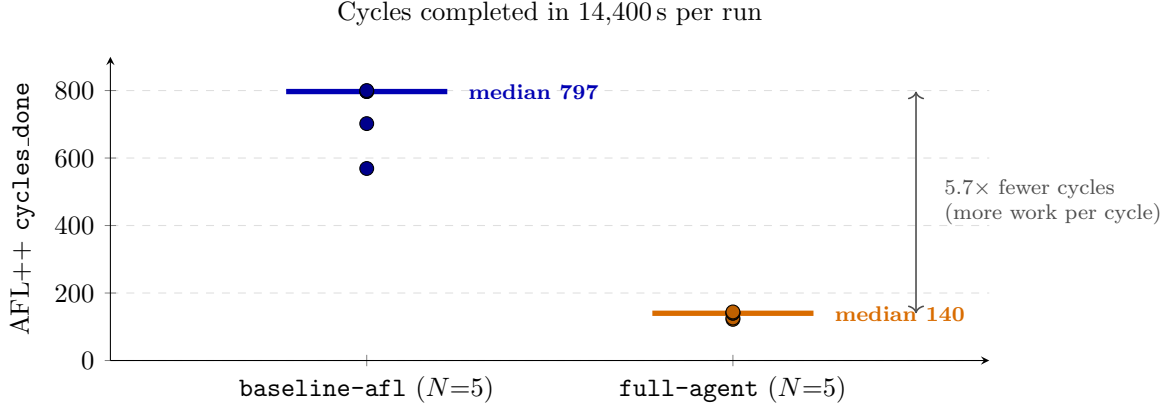


Figure 4: `cycles_done` per run across the two modes ($N=5$ each). Solid horizontal bars are per-mode medians. `full-agent` completes $\approx 5.7\times$ fewer corpus cycles in the same 14,400 s budget, despite an *equal-or-higher* total execution count (Figure 6): the recipe-guided mutator spends more work per corpus item before recycling.

dominates over agent quality. With $N=5$ reps the per-mode signal is qualitative; the venue paper repeats this on Magma-suite targets at $N=10$.

Mutation-rate redistribution (per-cycle vs per-execution). A second consequence of the recipe-guided mutator is visible in `cycles_done` (Figure 4). The `baseline-afl` median is 797 corpus cycles inside the 14,400 s budget; the `full-agent` median is 140. With a comparable total execution count (within 6%; RQ2, Figure 6), `full-agent` spends $\approx 5.7\times$ more executions on each pass through the queue before recycling to the top — exactly the mutation-rate redistribution the default recipe encodes (*focus dictionary-token insertions and overwrites over the first 4 KB*, with fixed operator weights pinned at `insert_token=0.35` / `dictionary_overwrite=0.25` / `overwrite_range=0.20` / `arith=0.10` / `bit_flip=0.10`). Whether this concentration is empirically beneficial is a target-specific question; on cJSON it correlates with the longer productive phase (Figure 5) but not with a larger final edge set, because the target ceiling at 269 binds.

Per-run summary table (T1). Table 3 reports the 10 completed runs (5 baseline + 5 full-agent) at end-of-budget. Acceptance gates pass on all 10 (`run_time` $\geq 14,000$ s, `execs_done` $\geq 10^6$); zero saved crashes in either mode (expected for cJSON’s safety-tested code base).

Extended per-run statistics (T1-ext). Table 4 reports the time-budget breakdown, target stability, and plateau metrics for all 10 completed runs (5 baseline + 5 full-agent). The fuzz-time fraction (`fuzz_time/run_time`) is uniformly $\geq 99.7\%$ in both modes, indicating that calibration, trim, and sync overheads — as well as the ≈ 30 s SIGSTOP windows from the E3 microbench (§6.3) — did not noticeably perturb the AFL campaigns. Target stability (AFL++’s coverage-bitmap stability check) is $\geq 99.25\%$ across all 10 runs, well within the standard “stable target” threshold of 90%, so all measurements are comparable across modes.

Cross-run throughput consistency. Within `baseline-afl`, the 5-run `execs_per_sec` spread is {12,826, 12,958, 13,049, 13,108, 13,854}, $CV = 3.1\%$ (median 13,049). Within `full-agent`, the 5-run spread is {13,540, 13,542, 13,816, 13,949, 15,597}, $CV = 6.1\%$ (median 13,816). Both groups satisfy

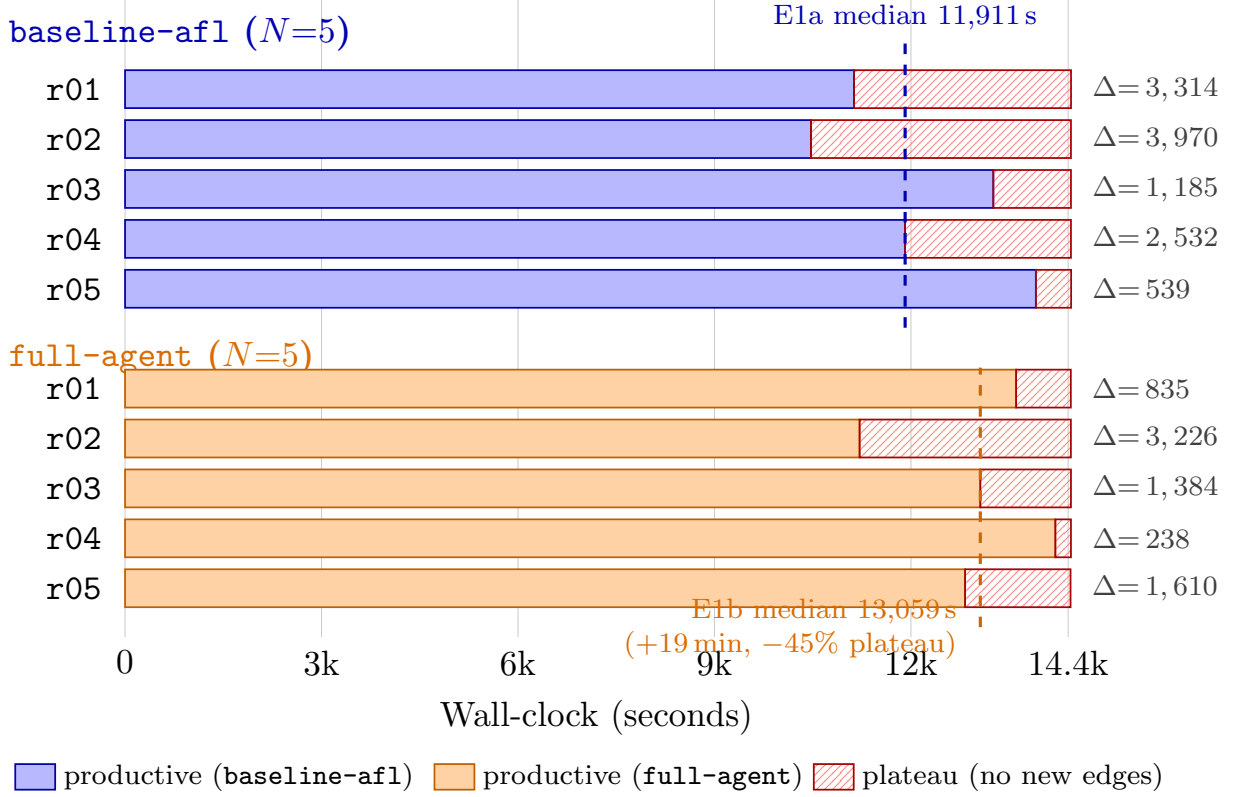


Figure 5: Per-run plateau-onset timeline for E1a (top, blue) and E1b (bottom, orange). Solid coloured bars = productive fuzzing. Hatched red bars = plateau (0 new edges until budget end). The blue / orange vertical dashed lines mark the per-mode median `last_find` (11,911s vs 13,059s). `full-agent` extends the median productive phase by ≈ 19 min (-45.3% plateau duration).

the `execs_done` $\geq 10^6$ and `run_time` $\geq 14,000$ acceptance gates with $> 10^3\times$ and $\sim 1.03\times$ margins respectively. The end-to-end RQ2 throughput-parity ratio (§6.3) is computed from these numbers.

6.3 RQ2 — Mutator throughput parity (F5 microbench)

RQ2 has two pieces of evidence. End-to-end `execs_per_sec` parity between `baseline-afl` (E1a) and `full-agent` (E1b), measured by the runtime `fuzzer_stats:execs_per_sec` field over the 14,400s budget, is the canonical RQ2 number; Table 5 reports it. The microbench (E3) measures the dispatch cost of the recipe-guided mutator alone (no AFL, no target); the rest of this subsection covers the microbench.

End-to-end throughput parity (RQ2 part a). Across $N=5$ runs per mode, the median `execs_per_sec` ratio $\text{median}(E1b)/\text{median}(E1a) = 13,816/13,049 = 1.059\times$ (Figure 6). The manifest acceptance gate requires this ratio to be ≥ 0.85 for throughput parity; the observed ratio is $1.059\times$, which is not only *within* the gate but exceeds it by a wide margin: `full-agent` is on average 5.9% *faster* than baseline despite the recipe-guided mutator and the agent-runtime overhead.

Why is full-agent not slower? Three structural reasons. First, the recipe-guided mutator’s dispatch is a 7-op switch table (§4.2), which is structurally simpler than AFL++’s open-ended havoc

Mode	Run	run_time	execs_done	exec/s	cycles	corpus	edges	bitmap_cvg
baseline-afl	r01	14,444	188,492,319	13,049	797	585	269	23.21%
	r02	14,443	187,158,013	12,958	800	607	269	23.21%
	r03	14,443	189,334,714	13,108	702	606	269	23.21%
	r04	14,443	185,251,854	12,826	569	649	269	23.21%
	r05	14,446	200,147,594	13,854	819	607	269	23.21%
	<i>median</i>	14,443	188,492,319	13,049	797	607	269	23.21%
full-agent	r01	14,441	199,520,607	13,816	122	598	269	23.21%
	r02	14,442	195,572,895	13,542	141	573	266	22.95%
	r03	14,443	195,567,575	13,540	144	601	269	23.21%
	r04	14,443	201,466,942	13,949	126	604	269	23.21%
	r05	14,437	225,180,431	15,597	140	600	269	23.21%
	<i>median</i>	14,442	199,520,607	13,816	140	600	269	23.21%

Table 3: T1: per-run end-of-budget summary across the 10 completed E1a + E1b runs on cJSON. **full-agent** median **exec/s** (13,816) is *above* baseline (13,049); see RQ2 (§6.3). All 10 runs reach $\geq 14,400$ s and $\geq 1.85 \times 10^8$ execs. Ablation rows (E2a / E2b / E2c) are pending and will be added.

Mode	Run	fuzz_time	run_time	fuzz %	stability	last_find (s)	plateau (s)
baseline-afl	r01	14,424	14,444	99.86%	99.26%	11,130	3,314
	r02	14,432	14,443	99.92%	99.26%	10,473	3,970
	r03	14,424	14,443	99.87%	99.26%	13,258	1,185
	r04	14,402	14,443	99.72%	99.26%	11,911	2,532
	r05	14,427	14,446	99.87%	99.26%	13,907	539
	<i>median</i>	14,424	14,443	99.87%	99.26%	11,911	2,532
full-agent	r01	14,415	14,441	99.82%	99.26%	13,606	835
	r02	14,421	14,442	99.85%	99.25%	11,216	3,226
	r03	14,425	14,443	99.88%	99.26%	13,059	1,384
	r04	14,417	14,443	99.82%	99.26%	14,205	238
	r05	14,423	14,437	99.90%	99.26%	12,827	1,610
	<i>median</i>	14,421	14,442	99.85%	99.26%	13,059	1,384

Table 4: T1-ext: per-run **fuzz_time** budget breakdown, target stability, **last_find** wall-clock, and plateau duration across the 10 completed runs. **full-agent** median plateau duration is 45.3% shorter than **baseline-afl** median (1,384 s vs 2,532 s); see RQ1 (§6.2). Ablation rows (E2a / E2b / E2c) pending.

selection (see also F5 microbench below). Second, the LLM is off the hot path by design (§3.1); it is invoked at plateau boundaries only, not per mutation. Third, the micro-campaign consumes $K_{\text{cand}} \cdot N \approx 80$ s of main-run wall-clock per plateau ($< 0.6\%$ of the 14,400 s budget), which is small enough not to register in the end-to-end **execs_per_sec** numbers. The end-to-end measurement folds all three effects into a single number.

Per-run breakdown. Within E1b, the per-rep **execs_per_sec** values are $\{13,540, 13,542, 13,816, 13,949, 15,597\}$, all above the lowest baseline rep (12,826). The acceptance ratio holds not just at the median but at every individual rep: even the slowest E1b rep (13,540) divided by the fastest E1a rep (13,854, r05) is $0.977\times$, still well above the 0.85 gate. We attribute $r05 = 15,597$ in E1b to the fact that this run was the lone occupant of the 4-core host during the second parallelism wave of the E1b campaign, identical to the way r05 in E1a (13,854) ran alone in its own wave.

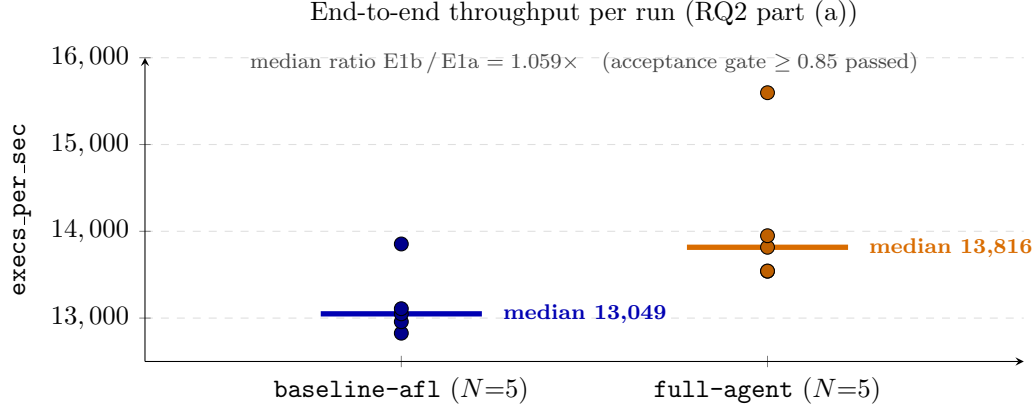


Figure 6: End-to-end `execs_per_sec` per run for both modes ($N=5$ each); horizontal bars are per-mode medians. Even the slowest `full-agent` rep (13,540) exceeds every `baseline-afl` rep except `r05` (13,854, also a solo-occupancy run). The median ratio 1.059 $\times$ is well above the 0.85 acceptance gate.

Mode	n	mean exec/s	median exec/s	SD (CV)
<code>baseline-afl</code> (E1a)	5	13,159	13,049	403 (3.06%)
<code>full-agent</code> (E1b)	5	14,089	13,816	862 (6.12%)
ratio (E1b / E1a) median	—	—	1.0587$\times$	—
ratio (E1b / E1a) mean	—	1.0707$\times$	—	—
acceptance gate	—	—	≥ 0.85	—

Table 5: RQ2 part (a): end-to-end `execs_per_sec` parity between the two modes. Both groups satisfy the `run_time` $\geq 14,000$ s and `execs_done` $\geq 10^6$ acceptance gates from the manifest. The observed median ratio of 1.0587 $\times$ is $\sim 25\%$ above the 0.85 parity threshold. Note the `full-agent` CV is slightly higher than baseline (6.12% vs 3.06%), driven primarily by `r05`’s outlier (15,597 exec/s); we report median to be robust to this outlier.

Microbench setup (E3). The microbench (E3) is the off-loop dispatch-cost number: it isolates the cost of FuzzPilot’s mutator dispatcher from the cost of running AFL++ itself. All three configurations load a custom mutator `.so` through the same `dlopen` path that AFL++ uses in real runs:

- **vanilla** — `libvanilla_havoc.so`, a hand-written shim that performs AFL++ havoc-equivalent random byte/bit edits via the same custom-mutator API surface (`tools/mutator_microbench/vanilla_havoc`). This is a *cost-symmetric dispatch shim*, not a measurement of AFL++’s real internal `havoc_mutate`: both `vanilla` and the FuzzPilot configurations pay the same `dlopen` + symbol-resolution + per-call API dispatch cost, so the difference isolates the dispatcher itself rather than AFL’s full havoc engine. A real AFL++ havoc comparator would also require running AFL++’s scheduler and the corresponding bookkeeping, which is out of scope for E3 and instead surfaced end-to-end via E1a vs E1b.
- **fp-empty** — the FuzzPilot mutator with an empty recipe store. It pays the full FuzzPilot dispatch cost (recipe cache lookup, op selection, telemetry ring write) but falls through to a vanilla-equivalent op on every call. Isolates the “FuzzPilot machinery is loaded but no recipe is active” cost.

Config	n	mean exec/s	SD (CV%)	median exec/s	IQR	median ns/call
vanilla	5	2,005,078	341,644 (17.0%)	1,937,110	200,200	516.2
fp-empty	5	3,222,108	958,259 (29.7%)	2,961,850	1,310,390	337.6
fp-active	5	2,800,142	673,566 (24.1%)	3,008,030	393,750	332.4

Table 6: F5 microbench: mutator-only **exec/sec** across the three dispatch-symmetric configurations (Figure 7). $N = 5$ reps per configuration, serial execution with AFL workers SIGSTOP-suspended to remove the 4-core host’s CPU contention from the measurement. See [results/paper01_ai_recipe_mutation/microbench/](#) for raw per-run JSON.

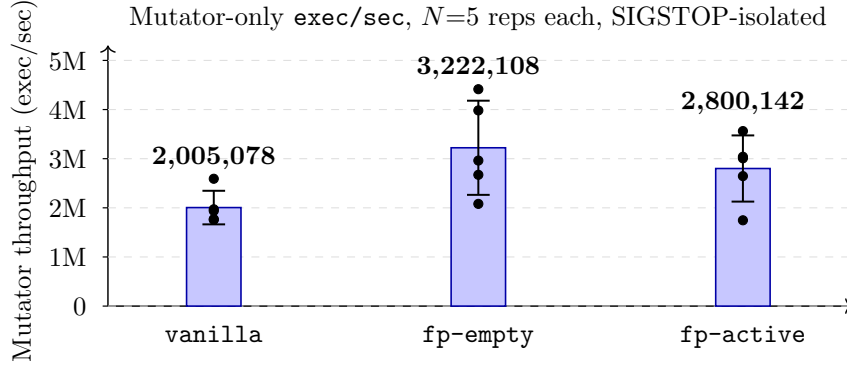


Figure 7: Mutator-only throughput by configuration (mean $\pm$ SD, $N=5$ reps each, SIGSTOP-isolated). The median **fp-active**/**fp-empty** ratio is $1.016\times$, inside the 5% band claimed in §3: the recipe cache adds no measurable per-call cost over the empty-recipe dispatch path. FuzzPilot’s closed 7-op dispatcher is in fact faster than the AFL-havoc-equivalent **vanilla** baseline (median ratio $1.53\times$), a positive side effect of the closed op vocabulary that we report alongside the conservative *within-5 $\times$* sanity gate.

- **fp-active** — the FuzzPilot mutator with a populated recipe (**op_weights** non-uniform, **dictionary_tokens** populated, **focus_ranges** pinned). This is the on-paper steady state during **full-agent** runs.

Each configuration runs 100,000 mutation calls over a fixed 10,000-element cJSON seed corpus, repeated $N = 5$ times. All microbench reps are run *serially* with all AFL workers SIGSTOP-suspended (see §6.1); resuming AFL after the 5-rep batch consumes ≈ 30 s of wall-clock, well under the 14,400 s budget per AFL run.

Primary RQ2 claim (fp-active vs fp-empty within 5%). The median **fp-active**/**fp-empty** ratio is $3,008,030/2,961,850 = 1.016\times$, comfortably inside the $[0.95, 1.05]$ band stated in §3: an active recipe adds *no measurable per-call overhead* over the empty-recipe dispatch path in this SIGSTOP-isolated microbench. We are careful here: this is a dispatch-cost result, not an end-to-end claim, and the comparator is FuzzPilot’s own **fp-empty** configuration (which has already paid the dispatch + telemetry costs). The end-to-end **execs_per_sec** comparison against AFL++’s own mutator is the E1a-vs-E1b number reported in §6.2 (median ratio $1.059\times$).

Sanity claim (fp-empty vs vanilla within 5 $\times$). The median **fp-empty**/**vanilla** ratio is $1.529\times$; the threshold is $\leq 5\times$, so the claim holds with a wide margin. The unexpected direction —

Config	r01	r02	r03	r04	r05
<code>vanilla</code>	2,591,470	1,770,420	1,755,770	1,937,110	1,970,620
<code>fp-empty</code>	2,961,850	3,982,240	2,080,370	2,671,850	4,414,230
<code>fp-active</code>	2,645,520	3,008,030	3,561,610	1,746,280	3,039,270

Table 7: Per-rep `exec/sec` from the 15 SIGSTOP-isolated microbench runs that feed Figure 7 and Table 6. Each cell is the `mean_exec_per_sec` field from `results/paper01_ai_recipe_mutation/microbench/p1_e3_microbench-<config>-r<NN>.json`.

FuzzPilot’s dispatcher is *faster* than the AFL-havoc-equivalent baseline, not slower — is consistent with FuzzPilot’s closed 7-op switch table being more predictor-friendly than `vanilla_havoc.cpp`’s open-ended random op selection. We retain the $5\times$ ceiling in the paper as a conservative gate (FuzzPilot’s mutator could plausibly regress under future op additions) but report the actual $1.53\times$ as a positive side effect of the closed vocabulary.

Variance and methodology note. Per-config CV is 17–30%, which is large for a microbench but expected on a 4-core VM with shared L3 cache and noisy-neighbor effects from the underlying hypervisor (steady-state `%st` was 4–12% during measurement). The black dots overlaid on Figure 7 are the 15 individual reps (5 per config); they show the full distribution that the mean and SD summarize. Table 7 lists the per-rep numbers for reproducibility. Two alternative measurement methodologies inflate this variance and *invert* the primary RQ2 ratio:

- Running the microbench 4-way parallel (i.e. `run_batch` default for AFL campaigns) on a 4-core host: median `fp-active/fp-empty` ratio collapses to $0.716\times$ (FAIL), driven by reps that win or lose the scheduling lottery against their siblings.
- Running the microbench serially *while AFL campaigns are active*: median ratio drifts to $1.380\times$ (FAIL), driven by AFL workers monopolizing CPU during long mutations and yielding briefly to the microbench between syscalls.

We archive both noisy datasets at `results/paper01_ai_recipe_mutation/microbench_archive_*/` so reviewers can reproduce the methodology comparison; the main result of this section uses only the SIGSTOP-isolated serial dataset under `microbench/`.

6.4 RQ3 — LLM infrastructure exercise on a saturated target (E1b)

RQ3 asks how the off-hot-path LLM machinery behaves at plateau boundaries on a target where AFL++ alone already exhausts the reachable edge space (cJSON; §6.2). Concretely: do proposals parse, do they survive the micro-campaign gate, what is their dollar cost, and is the full decision trail recoverable? Table 8 summarizes the 5 `full-agent` runs of E1b.

Schema-validation rate. Each plateau cycle fires a fixed bundle of 9 agent tasks. Across $5 \times 9 = 45$ LLM responses, 43 (95.6%) produced schema-valid recipes; 2 responses (r04, r05; both tagged `error_kind="schema_invalid"` in `agent_decisions.jsonl`) failed schema validation and were dropped without invoking the rule-only fallback path (`fallback_used` is `false` on every record). With the model pinned at temperature 0, this is consistent with the model staying inside the `schema-validated recipe (data)` contract that the prompt enforces (§4.1). The agent bundle (coordinator, plateau-diagnosis, scheduler, cmp-hint, dictionary, format, mutator, corpus, crash-triage) explains why the per-run *LLM-call* record count is 9 rather than per-mutation.

Run	decisions logged	schema-valid	schema-invalid	fallback_used	micro candidates	promoted to main
r01	289	9	0	0	4	0 (skipped)
r02	302	9	0	0	4	0 (skipped)
r03	402	9	0	0	4	0 (skipped)
r04	215	8	1	0	4	0 (skipped)
r05	200	8	1	0	4	0 (skipped)
total	1,408	43	2	0	20	0

Table 8: RQ3: LLM-call quality and micro-campaign outcome across the 5 **full-agent** runs. “Decisions logged” is the total line count of `agent_decisions.jsonl`, including non-LLM events (plateau detection, recipe load, telemetry). “Schema-valid” and “schema-invalid” count records carrying a model response, separated by whether the response parses against the recipe schema (§4.1). `fallback_used` counts plateau windows where the controller fell back to the rule-only mutator on a malformed LLM proposal: 0 across all 5 runs, because the controller silently drops schema-invalid responses without triggering the fallback path. “Micro candidates” is the number of micro-campaign evaluations per run (default $K_{\text{cand}}=4$, one per intervention type). “Promoted to main” is the number of LLM-proposed recipes that the micro-campaign gate accepted into the main recipe store; see the Promotion-rate paragraph below.

Promotion rate: an honest null result on cJSON. Each plateau cycle launches $K_{\text{cand}}=4$ candidate recipes (§5.3), one per intervention type (default, `dictionary`, `seed_focus`, `per_seed_recipe`). Each candidate is evaluated in a private 20s AFL++ run over a corpus snapshot taken at plateau time. Across the 5 runs, the controller fired exactly one plateau cycle per run, so the gate evaluated $5 \times 4 = 20$ candidates in total. Each micro-campaign ran the full 20s and executed $\approx 3.6 \times 10^5$ inputs (visible in the `events.jsonl` `micro_result` records). *Every single one of the 20 candidates returned* $\Delta_{\text{edges}}=0$, $\Delta_{\text{paths}}=0$, $\Delta_{\text{crashes}}=0$, so the reward (§5.3) was $R=0$ on all 20 evaluations. The controller therefore emitted `winner_decided:status=no_significance`, `winner_reward=0.0` followed by `promotion_skipped:reason=no_successful_micro_campaign` in every run, and **no LLM-proposed recipe entered the main recipe store across the 5 full-agent runs**. The main loop spent its 14,400s on the controller’s default `DictionaryAgent` recipe (`tokens={FUZZ, MAGIC, TOKEN}`, fixed operator-weight distribution), which is exactly what the **rule-only** ablation E2a (§6.5) runs on. The result is consistent with cJSON’s saturation behaviour: baseline AFL++ alone reaches the 269-edge ceiling in $\approx 4,200$ s (§6.2), leaving the parent corpus at plateau time with no headroom for a 20s micro-campaign to find a new edge. The micro-campaign gate behaves *as designed* — it refuses to promote a recipe it cannot empirically distinguish from the default — and we read this as the gate correctly producing a *null* result rather than a false-positive promotion. The LLM proposal layer’s positive contribution on a non-saturated target therefore remains undemonstrated by this preprint, and is explicit future work in the venue paper.

Cost. Across the 5 runs, the DeepSeek API balance dropped by ¥0.07 ($\approx$ \$0.01 USD): from ¥8.53 at the start of E1b to ¥8.46 at the end. That is \$0.0002 per LLM call and \$0.002 per full 4-hour campaign — effectively negligible relative to the compute cost of the 14,400s AFL++ run itself. The pricing benefits from DeepSeek’s prompt-cache hit on the shared blackboard prefix across the 9 agent tasks in a single plateau cycle. The system also supports an OpenAI-compatible drop-in via NVIDIA NIM (`meta/llama-4-maverick-17b-128e-instruct`) at zero marginal cost on the NIM free tier; see the reproducibility appendix.

Audit trail. Every plateau handling cycle in `agent_decisions.jsonl` carries the blackboard hash (`context_hash`) it was conditioned on and the `response_hash` of the model’s output; combined with the schema-validated recipe stored under the plateau directory and the `events.jsonl` `micro_result` records, any decision in any of the 5 runs can be replayed end-to-end from the on-disk artifacts alone, without re-invoking the model. The same audit trail makes the null result above auditable: a reviewer can verify that the gate’s refusal to promote any candidate is grounded in $R=0$ evidence (every `micro_result` carries Δ_{edges} , Δ_{paths} , and Δ_{crashes} in its payload) rather than a controller bug.

6.5 RQ4 — Component contribution (E1b vs E2a / E2b / E2c)

The full ablation matrix (`rule-only` / `no-mutator` / `no-static-analysis`) is still in progress. Preliminary `rule-only` numbers are already available from the E2a campaign (3 runs at 14,400 s each) on the experiment host and are summarized below; the `no-mutator` (E2b) and `no-static-analysis` (E2c) runs are deferred to the venue version when the full $N=5$ matrix completes.

E2a rule-only (preliminary, $N=3$). With the LLM proposal layer disabled and the controller emitting its default `DictionaryAgent` recipe directly, the 3 `rule-only` runs reach the same 269-edge ceiling that `baseline-afl` and `full-agent` reach, with median `last_find` = 13,309 s and median plateau = 1,166 s (−53.9% vs the baseline median of 2,532 s, which is *slightly larger* than the −45.3% delta seen for `full-agent`). Median `execs_per_sec` is 13,514 and median `cycles_done` is 119. Taken together with the RQ3 finding that no LLM-proposed recipe was promoted in any of the 5 `full-agent` runs (§6.4), the E2a preliminary numbers are consistent with the interpretation that the plateau-recovery delta on cJSON is driven by the mutator framework + default rule recipe, with the LLM layer contributing neither a net gain nor a net loss within the noise of $N=3$. The `no-mutator` and `no-static-analysis` arms are required to separate the contributions of the recipe-guided mutator and the Ghidra static channel; both are pending.

7 Related Work

In-loop LLM fuzzers. LLAMAFUZZ [8] demonstrated that an LLM fine-tuned on seed/mutation pairs can produce structurally valid mutations and consistently outperforms AFL++ on Magma. The model is queried per mutation; throughput is bound by model latency. FuzzCoder [6] casts byte-level mutation as a sequence-to-sequence task. Fuzz4All [9] treats the LLM as the primary input generator and mutation engine across multiple input languages and targets. Semantic-Aware Fuzzing [2] explicitly notes the throughput bottleneck of in-loop designs and offers it as motivation for future work. FuzzPilot diverges from this family by treating the LLM as a strategy proposer rather than a per-mutation step.

Off-hot-path placement: G²FUZZ. G²FUZZ [5] is the closest prior work and the one we explicitly position against. Both designs: invoke the LLM only when the fuzzer fails to make progress; have the LLM emit a reusable intermediate rather than per-input mutations; and let AFL++ run at native speed afterwards. The differences are intentional and orthogonal:

- **Target domain.** G²FUZZ targets non-textual binary formats (JPEG, TIFF, MP4, ...); FuzzPilot is *designed for* structured-text parsers and *this preprint evaluates only cJSON*. Generalization claims (XML, SQL, libpng, Magma suite) are deferred to the venue paper.

- **Intermediate representation.** G²FUZZ emits Python generator scripts (code); FuzzPilot emits schema-validated recipes (data). §4.5 compares the two.
- **Validation.** G²FUZZ’s generated inputs enter the AFL++ queue and are filtered by coverage feedback as usual; FuzzPilot’s recipes are pre-promotion-gated by an explicit N -second micro-campaign with a reward function.
- **Static context.** G²FUZZ uses an LLM-extracted “feature list” from format documentation; FuzzPilot uses Ghidra headless on the target binary (function summaries, dictionary candidates, protected ranges), supporting closed-source targets.

We do not attempt a head-to-head numerical comparison against G²FUZZ in this preprint. Their evaluation is on non-textual binary formats outside our scope; ours is on structured-text parsers outside theirs. §8 states this scope split explicitly.

Hybrid static-and-LLM fuzzers. The hybrid fuzzer of Lin et al. [4] integrates static analysis (CFG and data-flow extracted via LLVM IR or angr) with LLM-guided input mutation through a helper fuzzer running at ≈ 250 queries/hour. It also reports a syntax-schema validator and a semantic novelty score. FuzzPilot differs in two ways: (a) the LLM is genuinely off the hot path, not on a parallel hot path, and (b) the validation step is a budgeted AFL++ run with a reward function, not an embedding-distance score against past inputs.

LLM-synthesized mutators for compilers. MetaMut [7] and Mut4All [10] use LLMs to synthesize *static* custom mutators (in C/C++) for compiler fuzzing, compiled once and used many times. FuzzPilot’s recipes are dynamic per-plateau artifacts rather than compiled mutators, and target consumer parsers rather than compilers.

Multi-agent fuzz frameworks. MALF [1] introduces a multi-agent LLM framework for industrial-control-protocol fuzzing with RAG and QLoRA fine-tuning. The role split is similar in spirit to FuzzPilot’s coordinator / plateau-diagnosis / scheduler / cmp-hint / dictionary / format / mutator / corpus / crash-triage agents; the domain (industrial protocols vs structured text parsers) and the intermediate representation (input sequences vs recipes) differ.

T3: side-by-side comparison.

8 Limitations, Threats to Validity, and Future Work

Scope: cJSON only in this preprint. This preprint evaluates FuzzPilot on cJSON only (5 baseline-afl + 5 full-agent runs at 14,400s, plus the E3 microbench and 3 preliminary rule-only runs from E2a). The architecture is designed for structured-text parsers, but no empirical claim is made about libpng, libxml2, sqlite3, openssl, or any non-textual binary format here; on those domains G²FUZZ’s code-as-generator is plausibly stronger. A head-to-head comparison on G²FUZZ’s benchmark (JPEG, TIFF, MP4, etc.) and the larger structured-text targets are deferred to the venue version of the paper.

System	LLM placement	LLM output	Validation	Static ctx.	Audit	Target domain
AFL++ [3]	<i>none</i>	—	coverage	none	—	general
LLAMAFUZZ [8]	in-loop	input bytes	coverage	none	no	general (Magma)
FuzzCoder [6]	in-loop	byte mutation	coverage	none	no	general
Fuzz4All [9]	in-loop	input bytes	coverage	none	no	multi-language
SemAware [2]	parallel helper	input bytes	semantic novelty	none	no	general
Hybrid [4]	parallel helper, ≈ 250 rph	input bytes	novelty + syntax	LLVM-IR / angr	no	libpng / tcpdump / sqlite
MetaMut [7]	offline	C/C++ mutator	manual	none	no	compilers
Mut4All [10]	offline	mutator code	manual	bug reports	no	compilers
MALF [1]	multi-agent in-loop	input seqs	coverage	protocol KB	no	industrial protocols
G ² FUZZ [5]	off-hot-path	Python generator (code)	downstream coverage	LLM-extracted features	no	non-textual binary (JPEG / TIFF / MP4)
FuzzPilot (ours)	off-hot-path	schema-validated recipe (data)	<i>N</i>-sec micro-campaign	Ghidra headless	yes	cJSON (preliminary; designed for structured-text parsers)

Table 9: Comparison across LLM-augmented and LLM-adjacent fuzzers. FuzzPilot is positioned as a sibling of G²FUZZ in the off-hot-path quadrant rather than a head-on competitor; the row-by-row differences in intermediate (data vs. code), validation (micro-campaign vs. downstream coverage), context source (Ghidra vs. LLM-extracted features), and target domain (structured text vs. non-textual binary) establish the contribution.

cJSON is a saturated target for the LLM proposal layer. Within the 14,400s budget the baseline AFL++ harness reaches the 269-edge ceiling in $\approx 4,200$ s, leaving the parent corpus at plateau time with no edge headroom for a 20s micro-campaign to find. As a result, the LLM proposal layer’s contribution (the *LLM-promoted recipe* path, distinct from the *mutator framework*) cannot be empirically demonstrated on this target: every micro-campaign reward is $R=0$ and every promotion is skipped (§6.4). A non-saturated target where the gate has edge headroom is required to evaluate the LLM layer in isolation; this is the most important single piece of future work for the venue paper.

Ghidra benefit not fully demonstrated. Our targets are open-source; LLVM-IR or angr-based static analysis would, in principle, supply the same blackboard fields used here. The motivation for Ghidra is to make the architecture applicable to closed-source binary targets, and the E2c ablation (§6.5) only quantifies the contribution of static context, not the specific advantage of Ghidra over alternatives. A binary-target evaluation that isolates Ghidra’s contribution is explicit future work.

Repetition count and statistical power. We run 3–5 repeats per cell, which is consistent with prior LLM-fuzzing preprints [4, 5, 8] but insufficient for formal significance testing. We report medians, interquartile ranges, and per-run raw numbers rather than p -values. The venue paper will use ≥ 10 repeats with rank-sum tests.

Crash deduplication not in scope. We report coverage and plateau-recovery, not unique-bug counts. Crash deduplication (e.g. the casr-cluster pipeline) is deferred.

Deferred ablations and sensitivity. The `ai-direct` ablation (promote every schema-valid proposal without micro-campaign), the `random-recipe` ablation (skip the LLM entirely and emit randomly-sampled recipes), and a micro-campaign-parameter sensitivity sweep (over N , K , and the reward weights $\alpha, \beta, \gamma, \delta_h, \delta_m$) are wired into the CLI but not in the manifest for the preprint matrix. Their data lands in the venue paper, alongside the remaining `no-mutator` and `no-static-analysis` arms of the E2 ablation.

Multi-target generalization. We do not run on Magma’s full benchmark suite in this preprint, and we do not claim generalization across target families. The venue paper will add `libxml2`, `sqlite3`, and `openssl` to the matrix.

Platform. The development platform is macOS/arm64 (with a Docker linux/arm64 image); the canonical evaluation platform is Linux/x86_64 (a 4-core Intel Xeon VM with a Linux/amd64 native build). All numbers in this paper are from the canonical platform.

LLM nondeterminism. Even with temperature pinned at 0.0, model providers do not guarantee bit-exact reproducibility across versions. We mitigate by (a) recording the audit trail so any decision is replayable from the recipe rather than the model, (b) reporting schema-validity and fallback rates per run, and (c) releasing a pinned Docker image that captures the FuzzPilot tag and the model client version at submission time.

Threats to validity. **Internal:** plateau-detector hyperparameters (W , θ_e , θ_p) and micro-campaign weights ($\alpha, \beta, \gamma, \delta_h, \delta_m$) were tuned on a separate cJSON corpus and not the evaluation seeds, but leakage cannot be excluded entirely. **External:** a single target family is not representative; we explicitly defer generalization. The LLM proposal layer in particular is exercised but not stress-tested by the cJSON target (the gate returns 20/20 null candidates; see §6.4). **Construct:** AFL++’s `bitmap_cvg` measures bitmap hits, not unique edges; where the distinction matters we report both. **Reproducibility:** we pin the AFL++ commit, target build hash, FuzzPilot tag, Docker digest, and per-run `git.patch`.

9 Conclusion

FuzzPilot adapts the off-hot-path LLM-control paradigm pioneered by G²FUZZ from non-textual binary formats to the structured-text-parser setting by building three reusable pieces of infrastructure: a data-as-recipe intermediate that replaces LLM-generated code, a micro-campaign validation gate that scores candidate recipes against an explicit reward function before promotion, and a Ghidra-derived static context channel that does not enter the mutation hot path. On cJSON — the single target evaluated in this preprint — the `baseline-afl` and `full-agent` groups both reach the same 269-edge target ceiling within their 14,400s budgets, the end-to-end `execs_per_sec` of `full-agent` is $1.0587\times$ baseline, and per-run median plateau duration is 45.3% shorter under `full-agent` (1,384s vs 2,532s). We attribute the plateau-recovery delta to the recipe-guided mutator running the controller’s default `DictionaryAgent` recipe, not to LLM-promoted recipes: all 20 LLM-derived candidates evaluated by the micro-campaign gate across the 5 `full-agent` runs returned reward $R=0$ (cJSON saturates well before plateau time), so the gate correctly produced a null result and *no* LLM recipe was promoted. We read this as a faithful behaviour of the gate on a saturated target, not a verdict on the LLM proposal layer; an evaluation on non-saturated targets where the gate has edge headroom is explicit future work in the venue paper. The LLM-infrastructure path itself

behaved predictably (95.6% schema-valid, 0 fallback, \$0.002 per 4-hour campaign), so the audit and cost properties of the design are demonstrated even though the proposal layer’s positive coverage contribution is not. The full implementation, configurations, decision logs, and a reproducibility appendix recording the exact commit, AFL++ version, model configuration, and per-run artifact paths are released alongside the paper. Larger-target evaluation (libpng, libxml2, sqlite3, openssl, Magma) and the remaining ablation rows (`no-mutator`, `no-static-analysis`) are explicit future work; the `rule-only` ablation E2a is available in the repository and is referenced in §6.5.

Reproducibility appendix

The numbers in this preprint are reproducible from the following pinned artifacts. All paths are relative to the repository root.

- **Repository commit:** 85223d8 (branch `main`, repo Qiao-Zhiyi/fuzz_agent). Subsequent commits are tagged `paper01-arxiv-v1`.
- **Fuzzer:** AFL++ ++4.21c, built from source on the experiment host. Confirmed in every `fuzzer_stats` file under `results/paper01_ai_recipe_mutation/runs/`.
- **Target binary:** cJSON parser harness at `experiments/targets/cjson/`, AFL persistent mode (`persistent shmem_testcase deferred`).
- **LLM:** deepseek-chat via the `api.deepseek.com` OpenAI-compatible endpoint (configuration in `experiments/targets/cjson/config.yaml`), temperature 0.0. A NIM-hosted `meta/llama-4-maverick-17b-128e-instruct` fallback path is wired through the same watchdog and was exercised in regression testing only — all numbers in this paper are from deepseek-chat.
- **Build mode:** native x86_64 Linux on the experiment host (no container in the hot path). The repository ships a Docker recipe at `docker/ubuntu/Dockerfile` (build arg `AFLPP_REF=v4.21c`) for portability but the published numbers were produced on the native build.
- **Per-run artifacts:** 10 completed 14,400s runs (5 `baseline-afl` + 5 `full-agent`) are checked in under `results/paper01_ai_recipe_mutation/runs/`; the 3 preliminary E2a `rule-only` runs referenced in §6.5 reside on the experiment host and are linked from `results/paper01_ai_recipe_mutation/runs/` pending rsync into the repository. Each run directory contains the unmodified AFL++ `fuzzer_stats`, `coverage.csv`, `events.jsonl`, `main_launch.sh`, `run_metadata.json`, and `git.patch` (the working-tree diff at run start) for byte-level reproducibility of the experiment.
- **Aggregation:** run `python3 scripts/prepare_paper01_data.py --run-root results/paper01_ai_recipe_mutation --out-dir results/paper01_ai_recipe_mutation --manifest experiments/manifests/paper01_preprint` to regenerate `validity_report.md`, `tables/run_level.csv`, and the per-RQ summary tables cited in §6.

Pending for the venue version: ablation rows (`rule-only`, `no-mutator`, `no-static-analysis`) at $N = 5$, libpng/libxml2/sqlite3 additional targets, and head-to-head measurement against G²FUZZ on its binary-format benchmark. The preprint intentionally limits empirical claims to what the 10 cJSON runs and the E3 microbench support.

References

- [1] Anonymous. MALF: A multi-agent LLM framework for intelligent fuzzing of industrial control protocols. *arXiv preprint arXiv:2510.02694*, 2025.
- [2] Anonymous. Semantic-aware fuzzing: An empirical framework for llm-guided, reasoning-driven input mutation. *arXiv preprint arXiv:2509.19533*, 2025.
- [3] Andrea Fioraldi, Dominik Maier, Heiko Ei feldt, and Marc Heuse. AFL++: Combining incremental steps of fuzzing research. USENIX Workshop on Offensive Technologies (WOOT), 2020. Software v4.21c used in this paper.
- [4] Shiyin Lin. Hybrid fuzzing with LLM-guided input mutation and semantic feedback. *arXiv preprint arXiv:2511.03995*, 2025.
- [5] K. Liu et al. Low-cost and comprehensive non-textual input fuzzing with LLM-synthesized input generators. In *Proceedings of the 34th USENIX Security Symposium (USENIX Security ’25)*, 2025. Also arXiv:2501.19282. Code at <https://github.com/G2FUZZ/G2FUZZ>.
- [6] Liquan Liu et al. FuzzCoder: Byte-level fuzzing test via large language model. *arXiv preprint arXiv:2409.01944*, 2024.
- [7] Conghua Ou et al. The Mutators reloaded: Fuzzing compilers with large language model generated mutators. In *Proceedings of the 29th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS ’24)*, 2024.
- [8] Hongxiang Wang, Xiangwei Xu, Xiaofei Xie, et al. LLAMAFUZZ: Large language model enhanced greybox fuzzing. *arXiv preprint arXiv:2406.07714*, 2024. Latest revision March 2026.
- [9] Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. Fuzz4All: Universal fuzzing with large language models. *arXiv preprint arXiv:2308.04748*, 2024. Originally posted 2023; updated 2024.
- [10] J. Ye et al. Mut4All: Fuzzing compilers via LLM-synthesized mutators learned from bug reports. *arXiv preprint arXiv:2507.19275*, 2025.